

TASK5 AI 交易引擎：机器学习算法与场景应用

机器学习算法在量化交易领域的应用日益广泛，它能够帮助交易者从海量数据中挖掘有价值的信息，构建更精准的预测模型，从而优化交易策略。本任务通过使用 `scikit-learn` 库中的癌症诊断二分类数据集和股票财务指标收益数据集，建立和训练机器学习模型，并掌握混淆矩阵、AUC (Area Under Curve)、ROC (Receiver Operating Characteristic) 曲线等关键概念，用于评估模型的性能。

任务 1. 了解分类型机器学习算法，给出逻辑回归，决策树，随机森林等算法的解释

分类型机器学习算法 (classification) 主要用于处理输出变量为离散类别的问题，其目标是从样本特征中学习一个映射关系，将输入样本自动归入预先定义好的类别中。对于二分类任务，模型需要根据样本特征 x 预测其属于正类或负类的概率；对于多分类任务，模型需要在多个候选类别中输出最可能的类别。分类型算法在垃圾邮件识别、疾病诊断、用户流失预测、图像识别等场景中应用广泛。常见的分类型算法包括逻辑回归、决策树、随机森林、支持向量机、朴素贝叶斯和神经网络等。其中，逻辑回归强调线性可分或近似线性可分场景下的概率建模；决策树通过对特征空间进行递归划分来形成可解释的规则；随机森林则通过集成多棵决策树来降低单棵树的方差，提高模型泛化能力。

逻辑回归首先对输入特征做线性组合，得到打分函数 $z=w^T x+b$ ，再通过 Sigmoid 函数将该线性打分映射为类别为正类的概率：

$$p(y=1 \mid x)=\sigma(z)=\frac{1}{1+e^{-z}} . \quad (1)$$

由于输出的是概率，因此逻辑回归天然适合做分类解释。当概率大于某个阈值（常取 0.5）时判为正类，否则判为负类。训练逻辑回归时通常采用最大似然估计，其等价形式是最小化二元交叉熵损失，也就是对所有样本的负对数似然求和：

$$\mathcal{A}(w, b)=-\sum_{i=1}^n\left[y_i \log p_i+\left(1-y_i\right) \log \left(1-p_i\right)\right] \quad (2)$$

其中 $p_i=\sigma\left(w^T x_i+b\right)$ ，为了防止过拟合，常常还会加入正则化项，例如 L_2 正则：

$$\mathcal{A}_{reg}(w, b)=-\sum_{i=1}^n\left[y_i \log p_i+\left(1-y_i\right) \log \left(1-p_i\right)\right]+\lambda\|w\|_2^2 \quad (3)$$

逻辑回归的优点在于模型形式简单、训练高效、可解释性较强，并且输出概率便于阈值控制和业务决策；其局限性则在于表达能力偏弱，通常只能较好地刻画线性决策边界，若数据存在复杂非线性关系，则模型性能可能受限。

决策树是一类基于树结构的分类方法，其核心思想是通过一系列 if then 的条件判断，将样本逐步划分到更纯净的子节点中，最终在叶子节点输出类别。决策树的构建过程本质上是一个贪心的递归划分过程：在每个节点上，选择一个最优特征及其切分点，使得划分后的子节点纯度尽可能高。常见的纯度度量包括信息熵（最小化信息熵）、信息增益（最大化信息增益）和基尼系数（最小化基尼系数。一个方便记忆的方式是：基尼系数也用来衡量国家贫富差距，系数越大贫富差距越大。贫富差距的减小意味着样本纯度更高，因此是最小化基尼系数）。若某节点样本集合为 D ，其类别分布为 $p_k, k=1^K$ ，则该节点的信息熵定义为

$$H(D) = - \sum_{k=1}^K p_k \log p_k \quad (4)$$

对于按特征 A 划分后的子集 D_v ，信息增益可写为

$$Gain(D, A) = H(D) - \sum_{v=1}^V \frac{|D_v|}{|D|} H(D_v) \quad (5)$$

另一种常用指标是基尼系数。基尼系数定义样本类别的不平等程度，数值越高样本类别越混杂：

$$Gini(D) = 1 - \sum_{k=1}^K p_k^2 \quad (6)$$

划分时通常选择能够最大程度降低基尼指数的特征和切分点。

决策树的优点是结构直观、可解释性强，能够自然处理非线性关系和特征间的交互；缺点是单棵树容易过拟合，对数据扰动较敏感，往往需要剪枝或与集成方法结合来提升稳定性。

随机森林是在决策树基础上发展起来的集成学习（bagging）方法，其思想是多个弱模型投票，整体得到更强的模型。随机森林通常通过两个随机性来源来构建多棵树：第一，对训练集进行有放回抽样，得到不同的 bootstrap 样本；第二，在每个节点划分时，不从所有特征中选最优，而是仅从随机抽取的一部分特征中寻找最佳切分。这样做能够显著降低树与树之间的相关性，从而减小集成后的方差。设共有 T 棵树，每棵树输出类别预测 $h_t(x)$ ，则随机森林在分类任务中的输出通常采用多数投票：

$$\hat{y} = \text{mode}\{h_1(x), h_2(x), \dots, h_T(x)\} \quad (7)$$

如果模型输出的是类别概率，也可以取各树预测概率的平均值再进行判别：

$$\hat{p}(y=1 \mid x) = \frac{1}{T} \sum_{t=1}^T p_t(y=1 \mid x) \quad (8)$$

随机森林相比单棵决策树更稳定，泛化能力更强，通常不容易过拟合，而且对异常值和噪声有一定鲁棒性；但代价是模型可解释性下降，推理与训练开销也高于单棵树。总体而言，

逻辑回归、决策树和随机森林分别代表了线性概率模型、规则划分模型和集成树模型三种典型思路，它们构成了分类机器学习中的基础方法体系。

任务 2. 了解机器学习模型评价指标，给出混淆矩阵，AUC，ROC 等概念做出解释

在机器学习任务中，模型评价指标的作用是衡量模型预测结果与真实标签之间的一致程度，并据此比较不同模型或不同参数设置的优劣。对于分类任务而言，评价指标通常建立在**混淆矩阵（confusion matrix）**的基础之上。混淆矩阵将样本的真实类别与预测类别进行交叉统计，从而把预测结果分解为四种基本情况：真正例（True Positive, TP）表示真实为正类且被正确预测为正类的样本数；假正例（False Positive, FP）表示真实为负类但被误判为正类的样本数；真反例（True Negative, TN）表示真实为负类且被正确预测为负类的样本数；假反例（False Negative, FN）表示真实为正类但被误判为负类的样本数。混淆矩阵不仅能够直观反映模型的分类错误类型，还能进一步导出准确率、精确率、召回率、F1 值等一系列常用指标，因此它是分类模型评价的基础工具。

最常见的分类评价指标是**准确率（Accuracy）**，它表示所有预测正确的样本在总体样本中的比例，定义为 $\frac{TP+TN}{TP+TN+FP+FN}$ 。准确率在类别分布较均衡时具有较好的参考价值，但在类别极不平衡的场景下可能出现看似很高、实则无效的情况，例如负类样本占绝大多数时，即使模型几乎总是预测负类，准确率仍可能很高。因此，在不平衡分类问题中，通常还需要结合精确率（Precision）、召回率（Recall）和 F1 值进行综合评价。**精确率**关注预测为正类的样本中有多少是真正的正类，其定义为 $P=\frac{TP}{TP+FP}$ ；**召回率**关注真实正类样本中有多少被模型成功找回，其定义为 $R=\frac{TP}{TP+FN}$ 。精确率反映模型输出正类预测时的可靠程度，召回率反映模型识别正类的完整程度。**F1 值**则是精确率与召回率的调和平均，用于在二者之间取得平衡，其定义为 $F_1=\frac{2PR}{P+R}$ ，其中 P 表示精确率， R 表示召回率。一般来说，当业务更关注减少误报时，会更重视精确率；当业务更关注减少漏报时，会更重视召回率。

除了基于固定分类阈值的指标之外，ROC 曲线和 AUC 也是分类模型中非常重要的评价工具。**ROC（Receiver Operating Characteristic）**曲线描述的是当分类阈值不断变化时，模型在真正率（True Positive Rate, TPR ）与假正率（False Positive Rate, FPR ）之间的权衡关系。其中，真正率定义为 $TPR=\frac{TP}{TP+FN}$ ，本质上就是召回率；假正率定义为 $FPR=\frac{FP}{FP+TN}$ ，表示真实负类样本被误判为正类的比例。ROC 曲线以 FPR 为横轴、 TPR 为纵轴，将不同阈

值下得到的一系列点连接起来形成曲线。对于一个性能较好的模型而言，其 ROC 曲线会尽量靠近左上角，因为这意味着模型能够在保持较低误报率的同时获得较高的召回率。

AUC (Area Under the Curve) 是 ROC 曲线下面积的数值表达，通常用于衡量分类器的整体排序能力。AUC 的取值范围一般在 0 到 1 之间，数值越大表示模型越能把正样本排在负样本之前。AUC 具有较强的阈值无关性，即它不依赖于某一个固定分类阈值，而是综合评估模型在所有可能阈值下的整体表现。因此，当类别不平衡或阈值选择尚未确定时，AUC 往往比单一准确率更有参考价值。AUC 可以直观理解为随机抽取一个正样本和一个负样本，模型将正样本的预测分数排在负样本之前的概率。若 AUC 等于 0.5，说明模型的排序能力接近随机猜测；若 AUC 越接近 1，说明模型区分正负样本的能力越强。

综上所述，混淆矩阵提供了分类结果的基础统计信息，准确率、精确率、召回率和 F_1 值从不同角度刻画了模型的分类质量，而 ROC 曲线与 AUC 则进一步从阈值变化和排序能力的角度评估模型的整体性能。在实际应用中，评价指标的选择应结合任务目标、类别分布以及误报和漏报的代价共同确定，而不能只依赖单一指标做判断。

任务 3. python 编程实现以下内容：

- 1) 加载分类数据 (scikit-learn 中的乳腺癌数据集，因变量为 0 或 1)
- 2) 设定数据划分 (训练集与测试集)

Py 代码：

```
def load_breast_cancer_data(
    test_size: float = 0.2,
    val_size: Optional[float] = 0.1,
    random_state: int = 42,
) -> Tuple[np.ndarray, np.ndarray, np.ndarray, np.ndarray,
Optional[np.ndarray], Optional[np.ndarray]]:
```

"""加载乳腺癌数据集，并划分为训练集、验证集（可选）和测试集。

Parameters

test_size : float 测试集占比（默认 0.2）。

val_size : float or None 验证集占「训练+验证」的比例

random_state : int 保证可复现。

Returns

```

-----
X_train, X_test, y_train, y_test : np.ndarray 特征和标签。
X_val, y_val : np.ndarray or None 验证集特征和标签（若未划分则为 None）。
"""

data = load_breast_cancer()
X, y = data.data, data.target

# 先切出测试集
X_train_val, X_test, y_train_val, y_test = train_test_split(
    X, y, test_size=test_size, random_state=random_state,
    stratify=y
)

X_val, y_val = None, None
if val_size is not None and val_size > 0:
    # 从「训练+验证」中再切出验证集
    X_train, X_val, y_train, y_val = train_test_split(
        X_train_val, y_train_val, test_size=val_size,
        random_state=random_state, stratify=y_train_val,
    )
else: X_train, y_train = X_train_val, y_train_val
return X_train, X_test, y_train, y_test, X_val, y_val

```

3) 构建并训练分类模型，如决策树，随机森林等模型

Py 代码:

```

class BaseClassifier(ABC):
    """所有分类器的抽象基类。

    子类只需实现 ``_build_model()`` 即可获得统一的 ``fit / predict /
    predict_proba`` 接口。
    """

    def __init__(self, **kwargs: Any):
        self._model_kwargs = kwargs
        self._model: Any = None

    @abstractmethod
    def _build_model(self) -> Any:

```

```

        """构建底层模型对象（由子类实现）。"""
        ...

def fit(self, X: np.ndarray, y: np.ndarray) -> "BaseClassifier":
    """训练模型。"""
    self._model = self._build_model()
    self._model.fit(X, y)
    return self

def predict(self, X: np.ndarray) -> np.ndarray:
    """预测类别标签。"""
    self._check_fitted()
    return self._model.predict(X)

def predict_proba(self, X: np.ndarray) -> np.ndarray:
    """预测正类概率（shape = (n_samples, 2)）。"""
    self._check_fitted()
    return self._model.predict_proba(X)

def _check_fitted(self) -> None:
    if self._model is None:
        raise RuntimeError("模型尚未训练，请先调用 fit()。")

# 具体分类器实现
class LogisticRegressionClassifier(BaseClassifier):
    """逻辑回归分类器。"""

    def _build_model(self) -> LogisticRegression:
        return LogisticRegression(
            max_iter=self._model_kwargs.pop("max_iter", 5000),
            random_state=self._model_kwargs.pop("random_state", 42),
            **self._model_kwargs,
        )

class DecisionTreeClassifier(BaseClassifier):
    """决策树分类器。"""

    def _build_model(self) -> SkDecisionTree:

```

```

        return SkDecisionTree(
            random_state=self._model_kwargs.pop("random_state", 42),
            **self._model_kwargs,
        )

class RandomForestClassifier(BaseClassifier):
    """随机森林分类器。"""

    def _build_model(self) -> SkRandomForest:
        return SkRandomForest(
            random_state=self._model_kwargs.pop("random_state", 42),
            **self._model_kwargs,
        )

class XGBoostClassifier(BaseClassifier):
    """XGBoost 分类器。

    .. note::
        采用延迟导入：只有真正调用 ``_build_model()`` 时才会 import xgboost。
        若未安装或缺少系统库（如 libomp），将在此时抛出错误并给出提示。
    """

    def _build_model(self) -> Any:
        global _XGB_AVAILABLE
        try: from xgboost import XGBClassifier as XGBCls
        except ImportError:
            _XGB_AVAILABLE = False
            raise ImportError(
                "xgboost 未安装。请执行: pip install xgboost\n"
                "macOS 用户可能还需: brew install libomp"
            ) from None
        return XGBCls(
            eval_metric="logloss",
            random_state=self._model_kwargs.pop("random_state", 42),
            verbosity=self._model_kwargs.pop("verbosity", 0),
            **self._model_kwargs,
        )

```

4) 对建立的模型进行评估, 计算 AUC

Py 代码:

```
def evaluate_auc(  
    model: BaseClassifier,  
    X: np.ndarray,  
    y: np.ndarray,  
    ) -> float:  
    """计算模型在给定数据集上的 AUC。  
  
    Parameters  
    -----  
    model : BaseClassifier 已训练的分类器。  
    X : np.ndarray 特征。  
    y : np.ndarray 标签 (0/1) 。  
  
    Returns  
    -----  
    float ROC-AUC 值。  
    """  
    y_prob = model.predict_proba(X)[:, 1]  
    return float(roc_auc_score(y, y_prob))
```

表 1 不同分类器的 AUC

分类器	AUC
逻辑回归	0.9947
决策树	0.9107
随机森林	0.9911
XGBoost	0.9947

5) 画出 ROC 曲线

Py 代码:

```
def plot_roc_curve(
    models: Dict[str, BaseClassifier],
    X: np.ndarray,
    y: np.ndarray,
    save_path: Optional[str] = None,
    figsize: Tuple[int, int] = (8, 6),
) -> plt.Figure:
    """绘制一个或多个模型的 ROC 曲线。

    Parameters
    -----
    models : dict[str, BaseClassifier] 模型名称 → 已训练分类器的映射。
    X : np.ndarray 测试集特征。
    y : np.ndarray 测试集真实标签。
    save_path : str or None 若提供，则将图片保存到该路径（自动创建父目录）。
    figsize : tuple 图片尺寸。

    Returns
    -----
    matplotlib.figure.Figure
    """
    fig, ax = plt.subplots(figsize=figsize)

    for name, model in models.items():
        y_prob = model.predict_proba(X)[:, 1]
        auc_val = roc_auc_score(y, y_prob)
        fpr, tpr, _ = roc_curve(y, y_prob)
        ax.plot(fpr, tpr, lw=2, label=f"{name} (AUC = {auc_val:.4f})")

    # 对角线（随机分类器基线）
    ax.plot([0, 1], [0, 1], "k--", lw=1, alpha=0.5, label="Random (AUC = 0.5000)")

    ax.set_xlim(0.0, 1.0)
    ax.set_ylim(0.0, 1.05)
```

```

ax.set_xlabel("False Positive Rate", fontsize=12)
ax.set_ylabel("True Positive Rate", fontsize=12)
ax.set_title("ROC Curves – Breast Cancer Classification",
fontsize=14)
ax.legend(loc="lower right", fontsize=10)
ax.grid(True, alpha=0.3)
fig.tight_layout()

if save_path is not None:
    import os
    os.makedirs(os.path.dirname(save_path), exist_ok=True)
    fig.savefig(save_path, dpi=150, bbox_inches="tight")
    print(f"[INFO] ROC 曲线已保存至: {save_path}")

return fig

```

可以观察到，XGBoost、随机森林和逻辑回归具有显著更好的排序能力，决策树的排序能力略逊色但也远强于随机分类器。

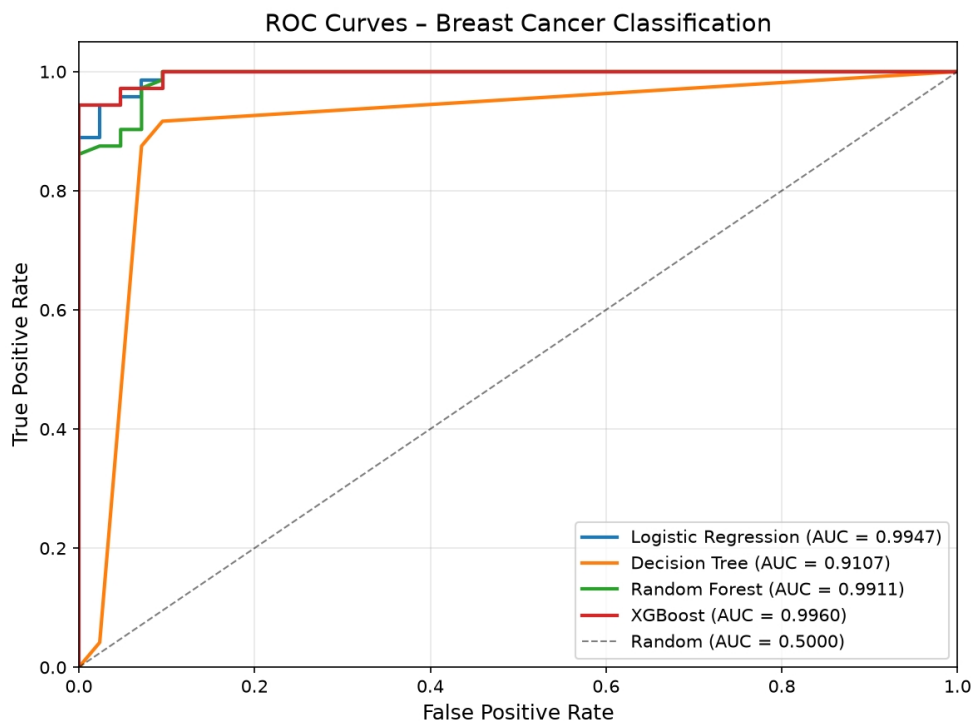


图1 不同分类器的 ROC 曲线